

Sputnik Devs

Skysride

Documentation

Team:

Cătălin Grigoriev – Engine Programmer

Tudor Baciuc – Level Designer

Ilie Beleniuc – 3D Environment Artist

Last update : 19.12.2025

Contents

I	Introduction	3
II	Features	3
III	Project structure.....	4
IV	System Requirements	5
V	Installation	5
V.1	Clone the repository	5
V.2	Build and run	5
V.3	Running	5
VI	Configuration	6
VII	Troubleshooting	6
VIII	Root-level directories	7
VIII.1	Properties/.....	7
VIII.2	Assets/	7
VIII.3	Documents/.....	7
VIII.4	Forms/.....	8
VIII.5	Interfaces/	8
VIII.6	Objects/.....	8
VIII.7	Packages/	9
VIII.8	Scenes/	9
VIII.9	Shaders/	9
VIII.10	Vendor/.....	10

I Introduction

Skystride is a C#/.NET project that use OpenTK (OpenGL bindings for .NET) to render graphics, load shaders, and run scene-based content.

II Features

The project follows a modular architecture with a clear separation of responsibilities, organized as follows:

- **Rendering**
Rendering is implemented using **OpenTK/OpenGL**, providing a flexible graphics pipeline for real-time rendering.
- **Shaders**
All **GLSL shader programs** are stored under the `shaders/` directory. This includes vertex, fragment, and other shader types necessary for rendering.
- **Scene Management**
Scene definitions and related data are maintained in the `scenes/` directory, supporting structured organization of multiple renderable environments.
- **Assets**
Project assets such as textures, models, and other resources are organized under the `assets/` directory for easy access and management.
- **Code Organization**
 - `objects/` contains core object implementations and entities used in the rendering system.
 - `interfaces/` defines abstract contracts and interfaces to ensure modularity and extendability across the codebase.
- **User Interface / Tooling**
Windows Forms-based tooling and UI components are located under the `forms/` directory, providing editors and interaction tools for the project.

This structure ensures a clear separation between graphics rendering, scene management, assets, core logic, and tooling, facilitating maintainability, scalability, and team collaboration.

III Project structure

The project is organized to separate concerns between core logic, rendering, assets, and tooling. Key components are as follows:

- **Program.cs**
Application entry point. Initializes the runtime and launches the main execution loop.
- **skystride.sln / skystride.csproj**
Visual Studio solution and C# project files, managing build configuration, references, and project metadata.
- **shaders/**
Directory containing **GLSL shader source files** used in the rendering pipeline (vertex, fragment, etc.).
- **scenes/**
Contains **scene definitions and logic**, including object placements, camera setups, and environmental configurations.
- **assets/**
Repository for project resources such as **textures, 3D models, and other asset files**.
- **objects/**
Core runtime objects and components used throughout the engine (entities, game objects, rendering objects, etc.).
- **interfaces/**
Abstractions and contracts to define modular and extendable systems within the engine.
- **forms/**
UI forms and tooling components, supporting editors and user interaction.
- **documents/**
Additional documentation and resources, such as design notes, diagrams, or technical references.
- **packages.config / packages/**
NuGet package references in legacy format for dependency management.

IV System Requirements

To build and run the project, the following system and software requirements must be met:

- **Operating System:**
Windows (recommended for full tooling support).
- **Development Tools:**
 - **Visual Studio** (recommended) or **msbuild** for building the project.
 - Ensure the version supports the **.NET Framework** / **.NET target** specified in `skystride.csproj`.
- **Graphics Hardware:**
 - **OpenGL-capable GPU** with up-to-date drivers, required for OpenTK/OpenGL rendering.
- **Dependencies:**
 - NuGet packages referenced in `packages.config` (legacy style) or restored automatically via the project file.

V Installation

V.1 Clone the repository

```
1. git clone https://github.com/catalingrigoriev285/skystride.git
2. cd skystride
```

1. Open `skystride.sln` in Visual Studio.
2. Restore NuGet packages (Visual Studio usually prompts automatically).

V.2 Build and run

1. Set the `skystride` project as Startup Project (if needed)

V.3 Running

On launch, the application should create a window and initialize OpenGL via OpenTK. If you see a blank screen or crash, check:

1. GPU driver/OpenGL support
2. Shader compilation errors (often logged to the console/output window)
3. Missing assets referenced from `assets/`

VI Configuration

The project uses configuration files to manage both general runtime settings and library-specific options:

- **App.config**
Contains **general runtime configuration**, including application-level settings, paths, and other customizable parameters used by the program at startup.
- **OpenTK.dll.config**
Stores **OpenTK-specific configuration**, such as runtime behavior, graphics context settings, or OpenGL-related options required by the OpenTK library.

These configuration files allow for **flexible adjustment of application behavior** without modifying the source code, supporting both development and production environments.

VII Troubleshooting

Missing NuGet Packages

If NuGet packages fail to restore automatically:

- **Visual Studio:** Right-click the solution in Solution Explorer → **Restore NuGet Packages**.
- **Command Line:** Run the following command from the solution directory:

```
nuget restore skystride.sln
```

- Ensure all packages listed in packages.config are restored to avoid build errors.

Shader Errors

Shader compilation or runtime errors may occur if shader files are not correctly located:

- Verify that all shader files in shaders/ are **copied to the output directory** or are loaded using the correct relative paths.
- If necessary, update the **build action** and **copy settings** in the project properties to ensure shaders are available at runtime.
- Check for syntax or version mismatches in GLSL shaders that may cause compilation failures.

VIII Root-level directories

VIII.1 Properties/

The Properties/ directory contains **typical Visual Studio/.NET project metadata** and configuration files, including:

- **AssemblyInfo.cs** – Defines assembly-level metadata such as version, title, description, and other attributes.
- **Resource and Settings Designer Files** – Auto-generated files depending on project type, managing embedded resources, application settings, and localization.

These files are essential for **project compilation, metadata management, and runtime configuration**.

VIII.2 Assets/

The assets/ directory contains **runtime resources** used by the project, including:

- **Textures** – Image files such as .png, .jpg, etc., used for materials, UI, and environment visuals.
- **Models** – 3D model files like .obj, .fbx, or other supported formats for scene objects.
- **Audio** – Sound files such as .wav, .mp3, or other audio assets used in scenes.
- **Miscellaneous Data Files** – Any additional resources required by scenes, scripts, or runtime systems.

Proper organization of this directory ensures **efficient asset management and reliable runtime loading**.

VIII.3 Documents/

The documents/ directory contains **non-code documentation and project artifacts**, including:

- **Design Notes, Specifications, and Diagrams** – Planning materials and technical schematics that support development.
- **Exported Documents** – PDFs, text files, or other formats containing reports, guides, or reference materials.
- **References and Instructions** – Additional resources such as setup instructions, tutorials, or workflow documentation relevant to the project.

This directory serves as a **central repository for all supplementary materials** that aid understanding, development, and maintenance of the project.

VIII.4 Forms/

The forms/ directory contains the **user interface (UI) layer**, typically implemented using **Windows Forms**. It generally includes:

- ***.cs** – Code-behind files containing form logic, event handling, and interaction with the underlying application systems.
- ***.Designer.cs** – Auto-generated files defining the UI layout and control properties.
- ***.resx** – Resource files associated with forms, such as images, localized strings, and other embedded assets.

This layer provides the **interactive UI components and tooling** for the application, separating presentation from core runtime logic.

VIII.5 Interfaces/

The interfaces/ directory contains **C# interfaces and abstractions** that define contracts used across the codebase. This includes:

- **I*.cs files** – Interface definitions such as `ISceneEntity`.
- **Contracts** – Specifications implemented by objects, scenes, and systems to ensure modularity, consistency, and extendability.

Using interfaces allows the project to **decouple implementation from design**, enabling flexible system integration, testability, and maintainable code.

VIII.6 Objects/

The objects/ directory contains **core runtime types** and components used throughout the engine. It typically includes:

- **Renderable Objects** – Meshes, cameras, lights, and other entities that participate in the rendering pipeline.
- **Math and Utility Wrappers** – Classes providing vector/matrix operations, transformations, and general-purpose helper functionality.
- **Game Objects / Engine Components** – Core entities and systems that define behavior, state, and interactions within the engine.
- **Asset Loaders / Resource Classes** – Utilities for loading and managing textures, models, shaders, and other resources at runtime.

This directory forms the **foundation of the engine's runtime logic**, encapsulating essential functionality and promoting modular, reusable code.

VIII.7 Packages/

The packages/ directory is used for **legacy NuGet package management**. It typically contains:

- **Downloaded Package Contents** – DLLs, build targets, and other files installed by NuGet to support project dependencies.

This folder allows the project to **reference external libraries** without requiring global installation.

VIII.8 Scenes/

The scenes/ directory contains **scene definitions and scene management logic**, including:

- **Individual Scenes / Levels** – Separate scene implementations such as menus, demo environments, or gameplay levels.
- **Scene Loader / Router / State Machine** – Systems responsible for loading, transitioning, and managing scene states.
- **Per-Scene Object Setup and Update Loops** – Initialization and update logic for objects specific to each scene, ensuring proper runtime behavior.

This directory provides the **structural foundation for scene management**, enabling organized control over environments, transitions, and object lifecycles within the engine.

VIII.9 Shaders/

The shaders/ directory contains **shader source code for OpenGL rendering**, including:

- **Vertex Shaders** – Files typically with extensions .vert, .vs, or .glsl, defining vertex transformations and attributes.
- **Fragment Shaders** – Files typically with extensions .frag, .fs, or .glsl, defining pixel-level color computations and effects.
- **Optional Shaders** – Depending on project needs, may include geometry shaders, compute shaders, or other specialized shader types.

Proper organization of this directory ensures that **shader programs are easily accessible, maintainable, and correctly integrated into the rendering pipeline**.

VIII.10 Vendor/

The vendor/ directory contains **the core engine code** and any **third-party dependencies not managed by NuGet**. It typically includes:

- **Engine Core** – All primary engine systems, runtime logic, and essential modules powering the application.
- **External Libraries** – Any third-party libraries included directly in the repository rather than via NuGet.
- **Utility Code or Native Binaries** – Helper classes, platform-specific binaries, or supplemental code required by the engine.

This directory serves as the **foundation of the project's runtime**, providing both the internal engine implementation and bundled external dependencies.